

AI ASSISTED CODING

ASSIGNMENT-10.3

NAME: DANCHANALA PAVAN

ROLL NO: 2403A52L07

BATCH:50

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):  
    result = 1  
    for i in range(1, n):  
        result = result * i  
    return result
```

Instructions:

- 1. Run the code and test it with factorial(5).**
- 2. Use an AI assistant to:**
 - o Identify the logical bug in the code.**
 - o Explain why the bug occurs (e.g., off-by-one error).**
 - o Provide a corrected version.**
- 3. Compare the AI's corrected code with your own manual fix.**
- 4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?**

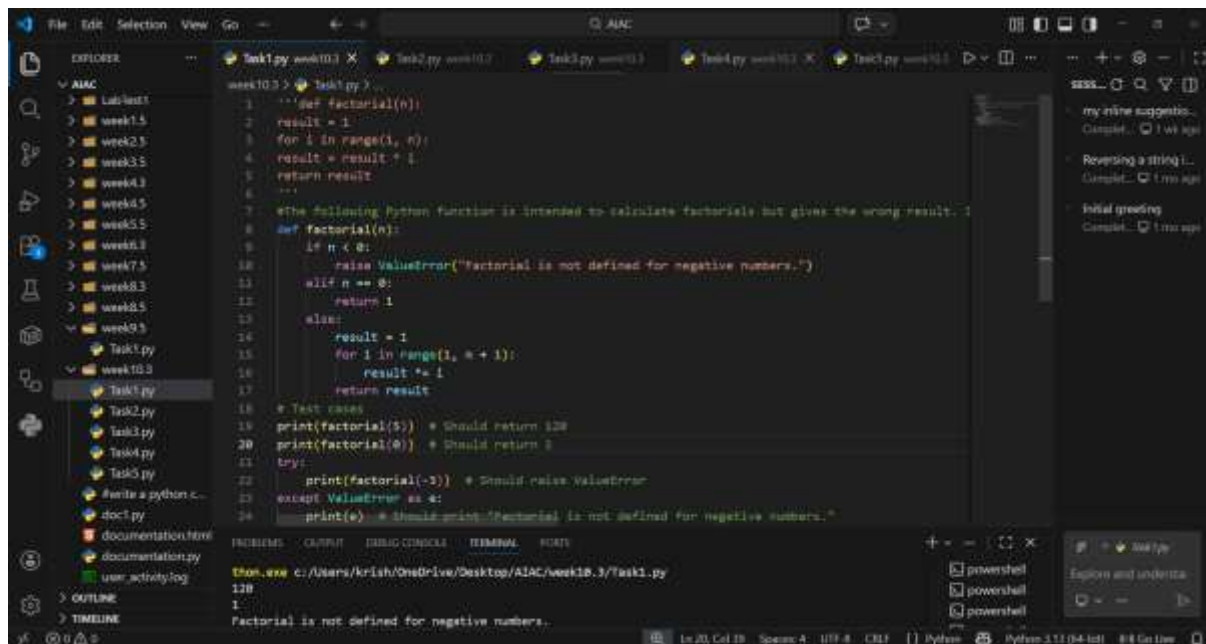
Expected Output:

Corrected function should return 120 for factorial(5).

PROMPT:

"The following Python function is intended to calculate factorials but gives the wrong result. Identify the bug, explain why it occurs, and provide a corrected version. Also, check if edge cases like zero or negative inputs are handled."

CODE AND OUTPUT:



The screenshot shows a VS Code editor with a Python file named `task1.py` in the `week10.3` directory. The code defines a `factorial` function with a loop that starts at `i = 1` and goes up to `n`. The function is tested with `factorial(5)`, `factorial(0)`, and `factorial(-1)`. The output shows that `factorial(5)` returns 120, `factorial(0)` returns 1, and `factorial(-1)` raises a `ValueError` with the message "Factorial is not defined for negative numbers.".

```
1 def factorial(n):
2     result = 1
3     for i in range(1, n):
4         result = result * i
5     return result
6
7 # The following Python function is intended to calculate factorials but gives the wrong result.
8 def factorial(n):
9     if n < 0:
10        raise ValueError("Factorial is not defined for negative numbers.")
11    elif n == 0:
12        return 1
13    else:
14        result = 1
15        for i in range(1, n + 1):
16            result *= i
17        return result
18
19 # Test cases
20 print(factorial(5)) # Should return 120
21 print(factorial(0)) # Should return 1
22 try:
23     print(factorial(-1)) # Should raise ValueError
24 except ValueError as e:
25     print(e) # Should print "Factorial is not defined for negative numbers."
```

PROBLEMS / OUTPUT / DEBUG CONSOLE / TERMINAL / PORTS

python.exe c:/Users/Krish/OneDrive/Desktop/AIAC/week10.3/task1.py

120
1
Factorial is not defined for negative numbers..

EXPLANATION:

This prompt explicitly asks the AI to detect the logical error (off-by-one in the loop), explain the reasoning, and fix it. It also ensures the AI considers edge cases beyond the immediate bug.

Problem Statement 2: Task 2 — Improving Readability & Documentation

Scenario: The following code works but is poorly written:

.

```
def calc(a, b, c):
```

```
if c == "add":
```

```
return a + b
```

```
elif c == "sub":
```

```
    return a - b
```

```
elif c == "mul":
```

```
    return a * b
```

```
elif c == "div":
```

Instructions:

5. Use AI to:

- o Critique the function's readability, parameter naming, and lack of documentation.

- o Rewrite the function with:

1. Descriptive function and parameter names.

2. A complete docstring (description, parameters, return value, examples).

3. Exception handling for division by zero.

4. Consideration of input validation.

6. Compare the original and AI-improved versions.

7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).

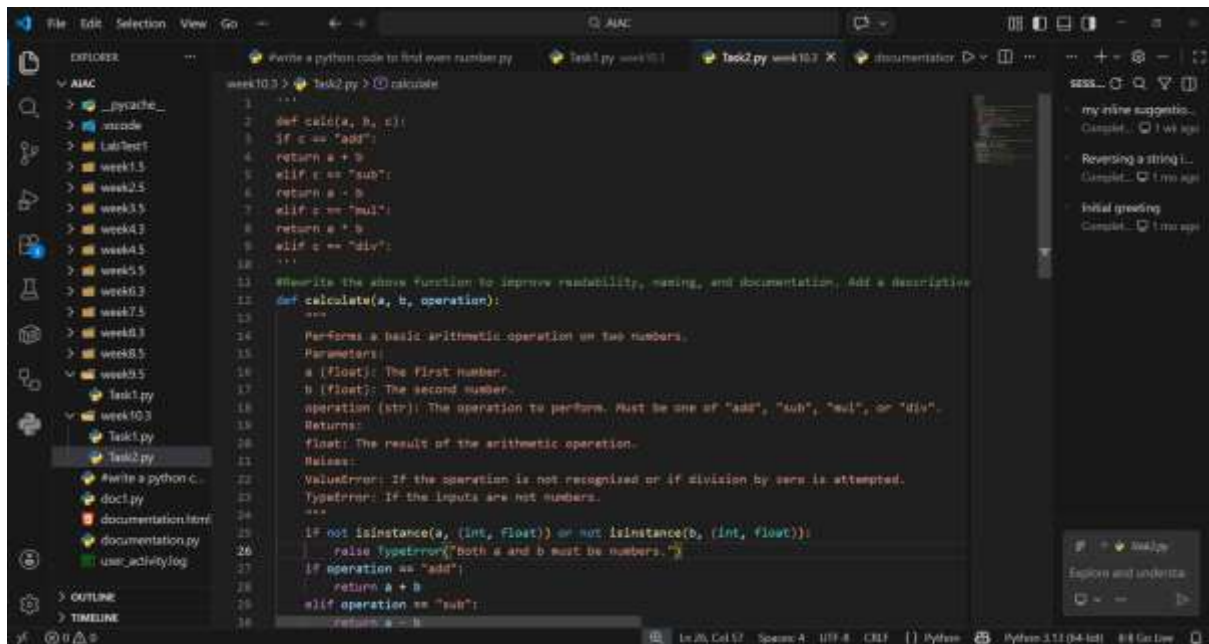
Expected Output:

A well-documented, robust, and readable function that handles errors gracefully.

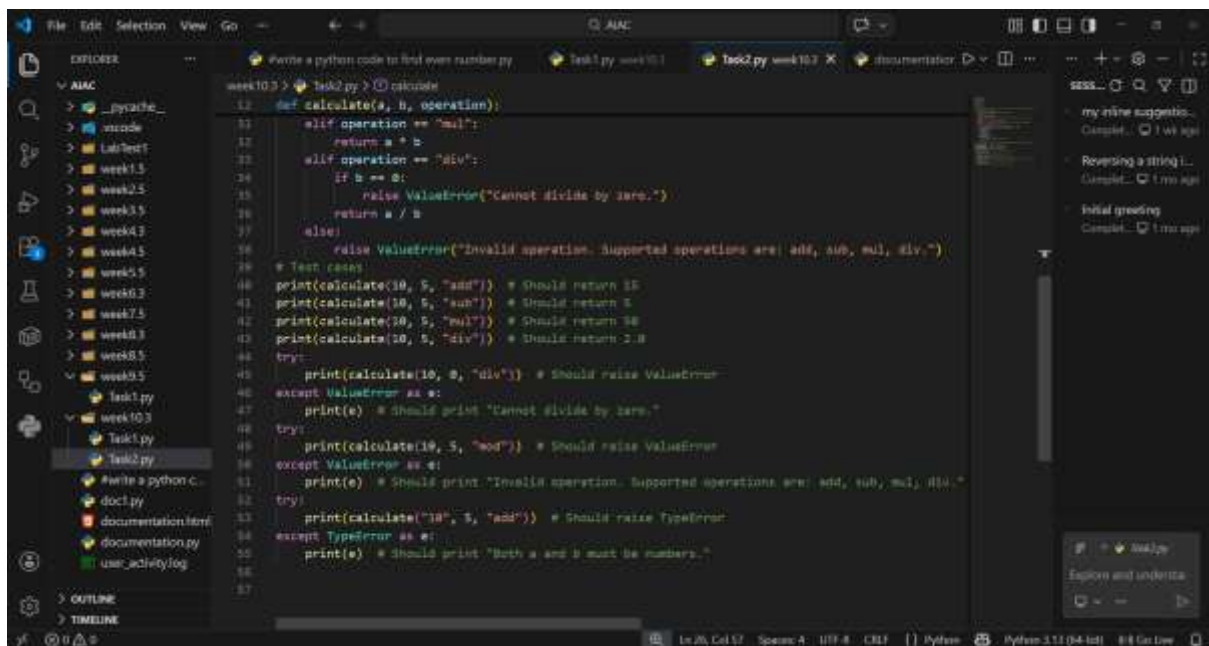
PROMPT:

"Rewrite the following function to improve readability, naming, and documentation. Add a descriptive docstring, handle division by zero, validate inputs, and make the code more robust."

CODE:

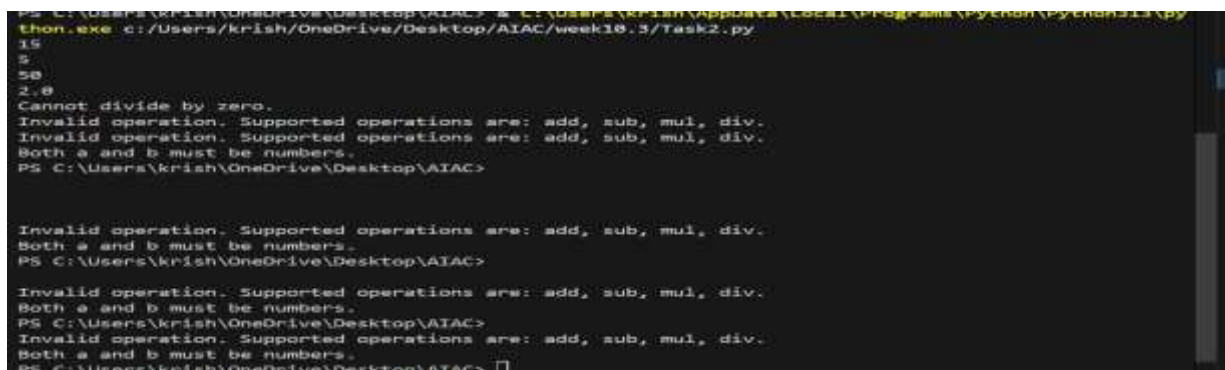


```
1  ...
2  def calc(a, b, c):
3      if c == "add":
4          return a + b
5      elif c == "sub":
6          return a - b
7      elif c == "mul":
8          return a * b
9      elif c == "div":
10         ...
11
12  #Rewrite the above function to improve readability, naming, and documentation. Add a descriptive
13  def calculate(a, b, operation):
14      """
15      Performs a basic arithmetic operation on two numbers.
16      Parameters:
17      a (float): The first number.
18      b (float): The second number.
19      operation (str): The operation to perform. Must be one of "add", "sub", "mul", or "div".
20      Returns:
21      float: The result of the arithmetic operation.
22      Raises:
23      ValueError: If the operation is not recognized or if division by zero is attempted.
24      TypeError: If the inputs are not numbers.
25      """
26      if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):
27          raise TypeError("Both a and b must be numbers.")
28      if operation == "add":
29          return a + b
30      elif operation == "sub":
31          return a - b
```



```
32  def calculate(a, b, operation):
33      ...
34      elif operation == "mul":
35          return a * b
36      elif operation == "div":
37          if b == 0:
38              raise ValueError("Cannot divide by zero.")
39          return a / b
40      else:
41          raise ValueError("Invalid operation. Supported operations are: add, sub, mul, div.")
42
43  # Test cases
44  print(calculate(10, 5, "add")) # Should return 15
45  print(calculate(10, 5, "sub")) # Should return 5
46  print(calculate(10, 5, "mul")) # Should return 50
47  print(calculate(10, 5, "div")) # Should return 2.0
48
49  try:
50      print(calculate(10, 0, "div")) # Should raise ValueError
51  except ValueError as e:
52      print(e) # Should print "Cannot divide by zero."
53
54  try:
55      print(calculate(10, 5, "mod")) # Should raise ValueError
56  except ValueError as e:
57      print(e) # Should print "Invalid operation. Supported operations are: add, sub, mul, div."
58
59  try:
60      print(calculate("10", 5, "add")) # Should raise TypeError
61  except TypeError as e:
62      print(e) # Should print "Both a and b must be numbers."
```

OUTPUT:



```
PS C:\Users\krish\OneDrive\Desktop\AIAC> python.exe c:\Users\krish\OneDrive\Desktop\AIAC\week10.3\Task2.py
15
5
50
2.0
Cannot divide by zero.
Invalid operation. Supported operations are: add, sub, mul, div.
Invalid operation. Supported operations are: add, sub, mul, div.
Both a and b must be numbers.
PS C:\Users\krish\OneDrive\Desktop\AIAC>

Invalid operation. Supported operations are: add, sub, mul, div.
Both a and b must be numbers.
PS C:\Users\krish\OneDrive\Desktop\AIAC>

Invalid operation. Supported operations are: add, sub, mul, div.
Both a and b must be numbers.
PS C:\Users\krish\OneDrive\Desktop\AIAC>

Invalid operation. Supported operations are: add, sub, mul, div.
Both a and b must be numbers.
PS C:\Users\krish\OneDrive\Desktop\AIAC>
```

EXPLANATION:

This prompt guides the AI to critique the original code and produce a professional, well-documented version with error handling. It ensures the AI doesn't just fix syntax but also improves clarity and usability.

Problem Statement 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer submits:

```
def Checkprime(n):  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True
```

Instructions:

- 8. Verify the function works correctly for sample inputs.**
- 9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to:**
 - o List all PEP8 violations.**
 - o Refactor the code (function name, spacing, indentation, naming).**
- 10. Apply the AI-suggested changes and verify functionality is preserved.**
- 11. Write a short note on how automated AI reviews could streamline code reviews in large teams.**

Expected Output:

A PEP8-compliant version of the function, e.g.:

```
def check_prime(n):
```

for i in range(2, n):

if n % i == 0:

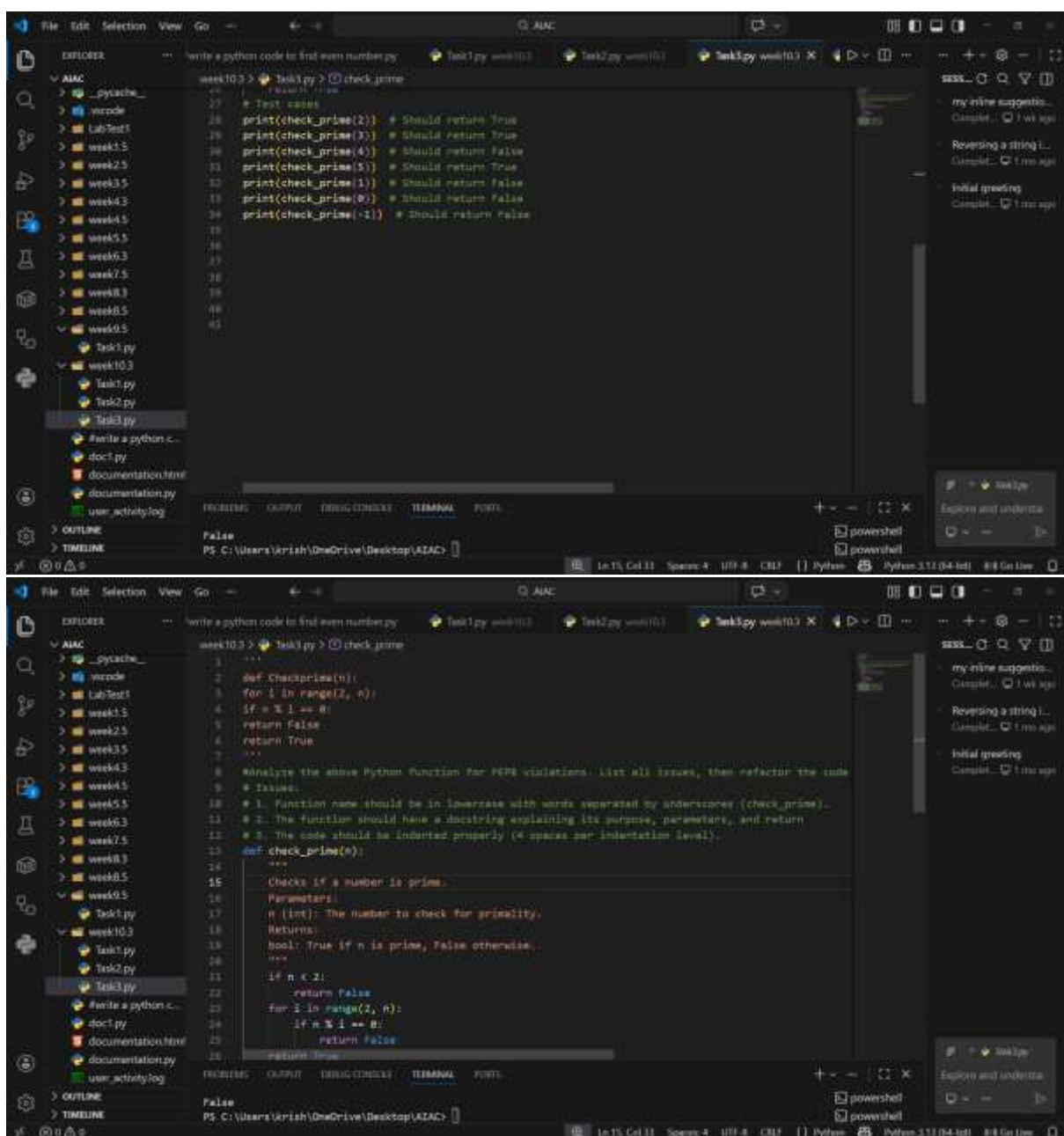
return False

return True

PROMPT:

"Analyze the above Python function for PEP8 violations. List all issues, then refactor the code to be fully PEP8-compliant while preserving functionality."

CODE:



OUTPUT:

```
PS C:\Users\krish\OneDrive\Desktop\AIAC> & C:\Users\krish\AppData\Local\Programs\Python\Python313\python.exe c:/Users/krish/OneDrive/Desktop/AIAC/week10.3/Task3.py
True
True
False
True
False
False
False
```

EXPLANATION:

This prompt ensures the AI acts like a linter, pointing out violations (naming, spacing, indentation) and then refactoring. It keeps the focus on compliance while maintaining correctness.

Problem Statement 4: AI as a Code Reviewer in Real Projects

Scenario:

In a GitHub project, a teammate submits:

```
def processData(d):
    return [x * 2 for x in d if x % 2 == 0]
```

Instructions:

1. Manually review the function for:

- o Readability and naming.
- o Reusability and modularity.
- o Edge cases (non-list input, empty list, non-integer elements).

2. Use AI to generate a code review covering:

- a. Better naming and function purpose clarity.
- b. Input validation and type hints.
- c. Suggestions for generalization (e.g., configurable multiplier).

3. Refactor the function based on AI feedback.

4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.

Expected Output:

An improved function with type hints, validation, and clearer intent,

e.g.:

from typing import List, Union

```
def double_even_numbers(numbers: List[Union[int, float]]) -> List[Union[int, float]]:
```

```
    if not isinstance(numbers, list):
```

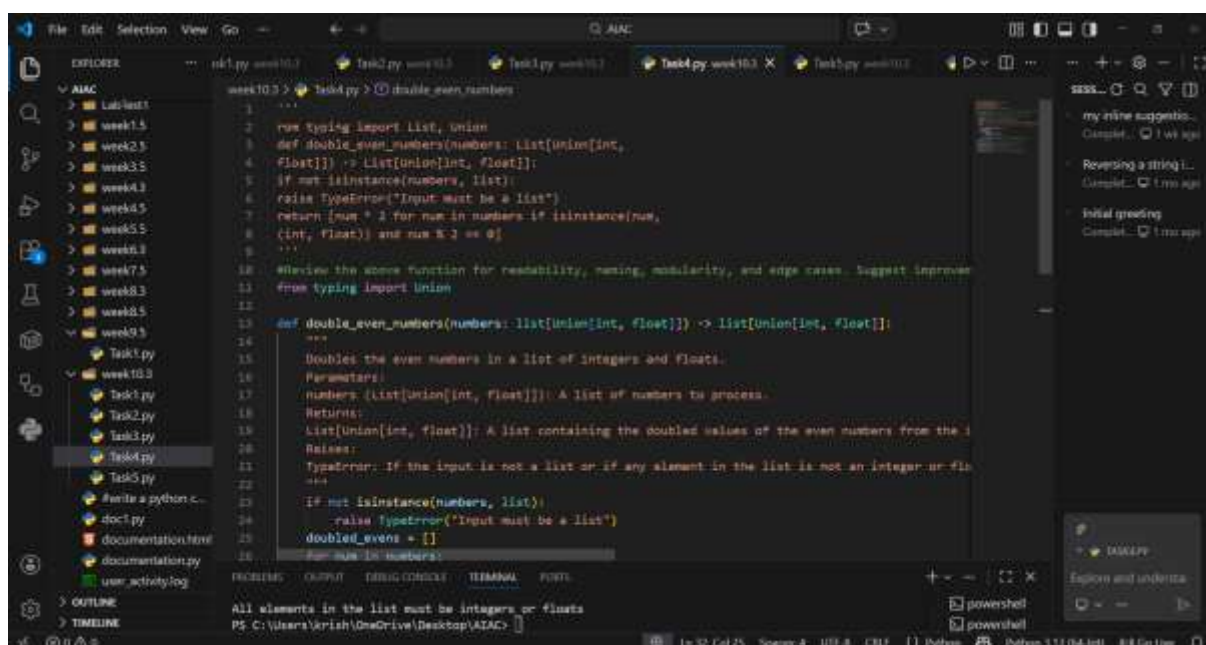
```
        raise TypeError("Input must be a list")
```

```
    return [num * 2 for num in numbers if isinstance(num, (int, float)) and num % 2 == 0]
```

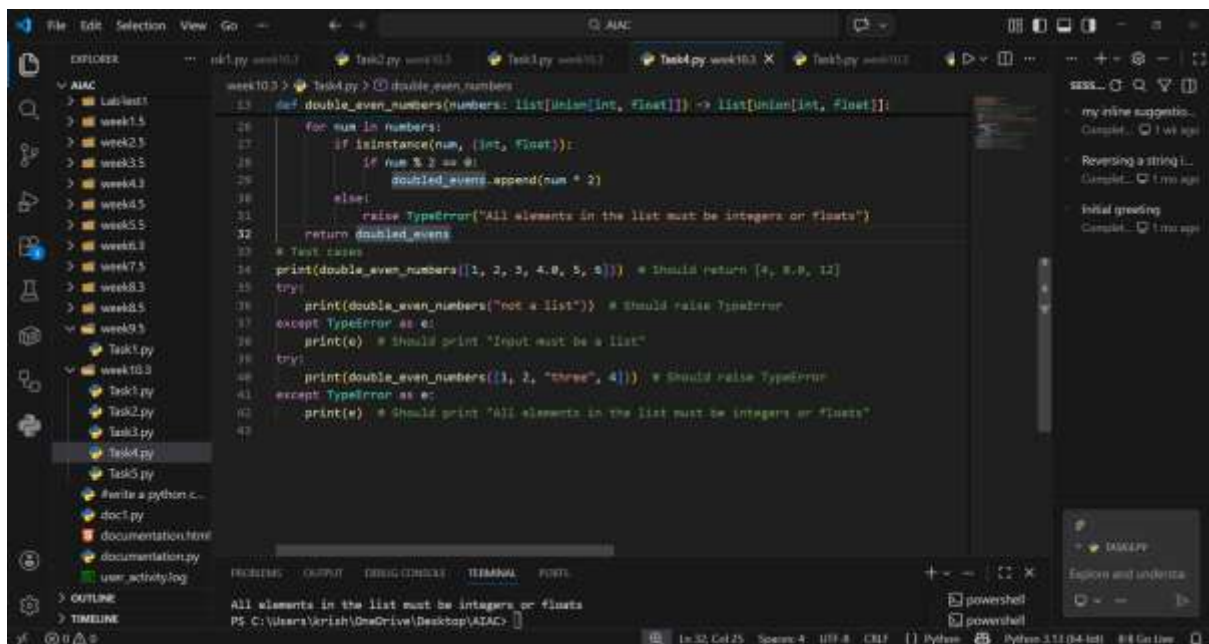
PROMPT:

“Review the above function for readability, naming, modularity, and edge cases. Suggest improvements with type hints, input validation, and generalization. Then refactor the function accordingly.”

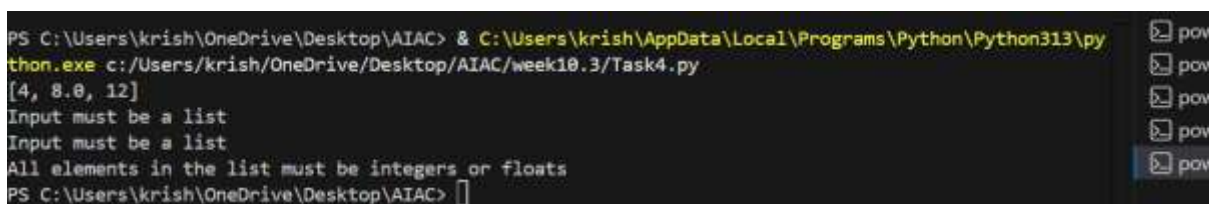
CODE:



```
1  ...
2  run typing import List, Union
3  def double_even_numbers(numbers: List[Union[int,
4  float]]) -> List[Union[int, float]]:
5  if not isinstance(numbers, list):
6  raise TypeError("Input must be a list")
7  return [num * 2 for num in numbers if isinstance(num,
8  (int, float)) and num % 2 == 0]
9  ...
10 #Review the above function for readability, naming, modularity, and edge cases. Suggest improve
11 from typing import Union
12
13 def double_even_numbers(numbers: List[Union[int, float]]) -> List[Union[int, float]]:
14     """
15     Doubles the even numbers in a list of integers and floats.
16     Parameters:
17     numbers (List[Union[int, float]]): A list of numbers to process.
18     Returns:
19     List[Union[int, float]]: A list containing the doubled values of the even numbers from the i
20     Raises:
21     TypeError: If the input is not a list or if any element in the list is not an integer or flo
22     """
23     if not isinstance(numbers, list):
24         raise TypeError("Input must be a list")
25     doubled_evens = []
26     for num in numbers:
```

OUTPUT:



EXPLANATION:

This prompt positions the AI as a reviewer, not just a fixer. It ensures the AI provides critique, suggests enhancements, and produces a refactored version with clearer intent and robustness.

Problem Statement 5: — AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

```
def sum_of_squares(numbers):
```

```
    total = 0
```

```
    for num in numbers:
```

```
        total += num ** 2
```

```
    return total
```

Instructions:

1. Test the function with a large list (e.g., `range(1000000)`).

2. Use AI to:

- o Analyze time complexity.

- o Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable).

- o Provide an optimized version.

3. Compare execution time before and after optimization.

4. Discuss trade-offs between readability and performance.

Expected Output:

An optimized function, such as:

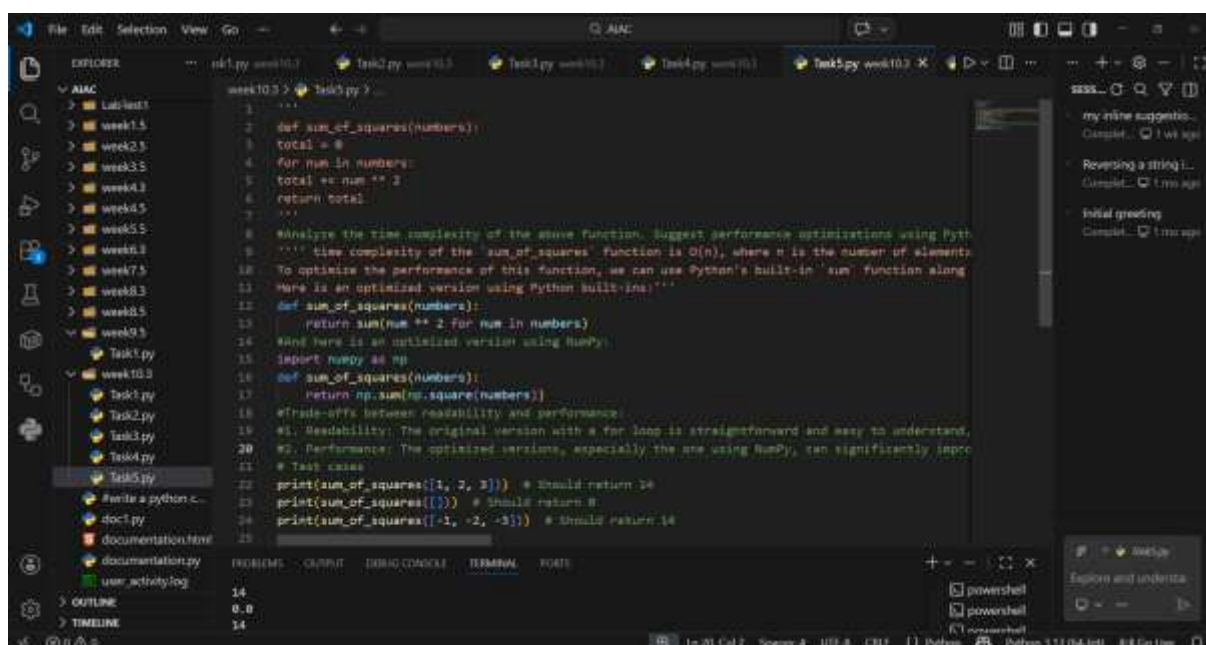
```
def sum_of_squares_optimized(numbers):
```

```
    return sum(x * x for x in numbers)
```

PROMPT:

"Analyze the time complexity of the above function. Suggest performance optimizations using Python built-ins or libraries like NumPy. Provide an optimized version and discuss trade-offs between readability and performance."

CODE AND OUTPUT:



```
1  ... Task5.py ...
2  def sum_of_squares(numbers):
3      total = 0
4      for num in numbers:
5          total += num ** 2
6      return total
7
8  """Analyze the time complexity of the above function. Suggest performance optimizations using Python
9  built-ins or libraries like NumPy. Provide an optimized version and discuss trade-offs between readability
10  and performance.
11  """
12  def sum_of_squares_optimized(numbers):
13      return sum(num ** 2 for num in numbers)
14
15  """And here is an optimized version using NumPy.
16  """
17  import numpy as np
18  def sum_of_squares_numpy(numbers):
19      return np.sum(np.square(numbers))
20
21  # Trade-offs between readability and performance:
22  # 1. Readability: The original version with a for loop is straightforward and easy to understand.
23  # 2. Performance: The optimized versions, especially the one using NumPy, run significantly faster.
24  # Test cases
25  print(sum_of_squares([1, 2, 3])) # Should return 14
26  print(sum_of_squares([0])) # Should return 0
27  print(sum_of_squares([-1, -2, -3])) # Should return 14
```

EXPLANATION:

This prompt ensures the AI evaluates efficiency, proposes alternatives, and explains trade-offs. It balances optimization with maintainability, which is crucial in real-world projects.